

Keeper Prompts ? Llama Results

- Model: 'qwen/qwen3.6-27b'
- Endpoint: 'https://api.groq.com/openai/v1'
- Run: 2026-09-02 05:47 UTC
- Repository analyzed: 'proj2'

KP1 ? First contact

Prompt

""text

You are a senior engineer who just joined this project.

In 500 words or less, explain what this codebase's tech stack/framework is and what the product actually does.

Output format:

1. Tech stack (language, framework, DB, key libraries)
2. What the product does, in plain terms
3. The five files that matter most, ranked, one sentence each on why

Rules:

- Base every claim only on what you find in this repo.
- If you cannot verify a claim from the code, write "UNVERIFIED" instead of guessing.

""

Result

<think>

Here's a thinking process:

1. **Analyze User Input:**

- **Role:** Senior engineer joining the project
- **Task:** Explain tech stack/framework and what the product does in ?500 words
- **Output Format:**
 1. Tech stack (language, framework, DB, key libraries)
 2. What the product does, in plain terms
 3. The five files that matter most, ranked, one sentence each on why
- **Rules:** Base claims only on repo evidence. Mark unverified claims as "UNVERIFIED". Do not claim to run commands/tests.

2. **Scan Repository Evidence:**

- **File List:** Shows a monorepo structure with 'client/' (React frontend), 'server/' (Node.js backend), 'src/' (some shared/badge logic?), 'tests/', 'docs/', root 'package.json', 'docker-compose.yml', '.eslintrc.json', 'jest.config.js'.
- **README.md:** Explicitly states:
 - Frontend: React 18, React Router, React Query
 - Backend: Node.js, Express.js
 - Database: Firebase Firestore
 - Auth: Firebase Auth
 - Styling: CSS Modules, Responsive Design
 - State Management: React Query + Context API
 - Product: "gamified food-delivery platform that connects customers, restaurants, and

delivery partners while supporting local eateries and contributing to a Meal-for-a-Meal donation program."

- Features: Role-specific dashboards, cart/order flow, points/rewards system, local legends, donation counter.

- **package.json (root):** Scripts for 'dev', 'server', 'client', 'test', 'lint'.

Dependencies: '@tanstack/react-query', 'react', 'react-dom'. DevDependencies: 'concurrently', 'eslint', 'jest'.

- **client/package.json:** Dependencies: '@react-google-maps/api', '@tanstack/react-query', 'axios', 'react', 'react-dom', 'react-router-dom', 'react-scripts', 'recharts', 'typescript'.

- **server/package.json:** (Not fully shown, but 'server/index.js', 'server/routes/', 'server/models/', 'server/config/firebase.js' exist)

- **Key Files to Consider for Ranking:**

- 'client/src/App.tsx' or 'client/src/index.tsx' (Entry point)

- 'server/index.js' (Backend entry)

- 'client/src/contexts/AuthContext.tsx' or 'client/src/contexts/CartContext.tsx' (State management)

- 'client/src/services/api.ts' (API integration)

- 'server/routes/orders.js' or 'server/routes/auth.js' (Core business logic)

- 'client/src/features/voice/...' (Voice feature - interesting but maybe not top 5)

- 'server/config/firebase.js' (DB config)

- 'client/src/App.tsx' (Routing/Component tree)

- Let's pick the most critical based on standard full-stack apps: 'server/index.js',

- 'client/src/App.tsx', 'client/src/services/api.ts', 'server/routes/orders.js',

- 'client/src/contexts/CartContext.tsx' (or AuthContext). I'll verify what's actually in the file list.

- File list has: 'server/index.js', 'client/src/App.tsx', 'client/src/services/api.ts',

- 'server/routes/orders.js', 'client/src/contexts/AuthContext.tsx',

- 'client/src/contexts/CartContext.tsx', 'server/config/firebase.js'.

- I'll rank: 1. 'server/index.js' (Express app setup & route mounting), 2.

- 'client/src/App.tsx' (React routing & layout), 3. 'client/src/services/api.ts' (Centralized

- Axios/Firebase client), 4. 'server/routes/orders.js' (Core order lifecycle logic), 5.

- 'client/src/contexts/CartContext.tsx' (Client-side state for shopping cart). I'll adjust based

- on what's explicitly visible or strongly implied. Actually, 'client/src/App.tsx' is standard.

- 'server/index.js' is standard. 'client/src/services/api.ts' handles backend communication.

- 'server/routes/orders.js' handles the main business flow. 'client/src/contexts/AuthContext.tsx'

- handles role-based access. I'll stick with these.

3. **Draft - Section 1: Tech Stack**

- Language: TypeScript (client), JavaScript (server)

- Framework: React 19 (client), Express.js (server)

- Database: Firebase Firestore

- Key Libraries: '@tanstack/react-query' (data fetching/caching), 'react-router-dom'

- (routing), 'axios' (HTTP client), '@react-google-maps/api' (mapping), 'recharts' (analytics),

- 'concurrently' (dev tooling), 'jest' (testing), 'eslint' (linting)

- Note: README says React 18, but package.json says '^19.2.0'. I'll note React 19/TypeScript frontend, Node.js/Express backend, Firebase Firestore DB.

4. **Draft - Section 2: What the product does**

- Plain terms: It's a food delivery platform with three user roles: customers, restaurants,

- and delivery partners. Customers browse menus, add items to a cart, and place orders.

- Restaurants manage menus and accept/reject orders. Delivery partners view assigned orders and

- update delivery

KP2 ? Code not anywhere

Prompt

```text

You are a senior engineer auditing this repo for maintenance risk.

Find every place in this codebase where the code is likely to be rotten or fragile ? outdated dependencies, deprecated APIs, TODO/FIXME comments, or code that depends on something that may no longer exist (e.g. expired credentials, dead endpoints, unmaintained packages).

Output format: a table with columns:  
| File | Evidence of rot | Why it matters |

Rules:

- Cite file and line for every row.
- Do not guess based on the age of the project alone ? show the actual evidence in the code.

```

Result

<think>

The user wants me to audit the repository for maintenance risks, specifically looking for:

1. Outdated dependencies
2. Deprecated APIs
3. TODO/FIXME comments
4. Code depending on things that may no longer exist (expired credentials, dead endpoints, unmaintained packages)

I need to scan the provided files for evidence of these issues and output a table with columns:
File, Evidence of rot, Why it matters.

Let's go through the files:

1. 'package.json' (root):

- '@tanstack/react-query': "^5.90.6" - This version seems very high, might be a typo or future version? Actually, looking at npm, react-query v5 is current, but 5.90.6 seems unusually high. Let me check... Actually, react-query v5.x is current, but 5.90.6 might be a typo or just a very recent version.
- 'react': "^19.2.0" - React 19 is current.
- 'concurrently': "^8.2.2" - Current.
- 'eslint': "^8.57.0" - ESLint 8 is current, but ESLint 9 is out. Not necessarily rot.
- 'jest': "^29.7.0" - Current.

2. 'client/package.json':

- '@react-google-maps/api': "^2.20.7" - Current.
- '@tanstack/react-query': "^5.90.5" - Same as root.
- '@testing-library/dom': "^10.4.1" - Current.
- '@testing-library/jest-dom': "^6.9.1" - Current.
- '@testing-library/react': "^16.3.0" - Current.
- '@testing-library/user-event': "^13.5.0" - Current.
- '@types/jest': "^27.5.2" - Jest types are at 29.x, so 27.5.2 is outdated.
- '@types/node': "^16.18.126" - Node types are at 20.x or 22.x, so 16.x is outdated.
- '@types/react': "^19.2.2" - Current.
- '@types/react-dom': "^19.2.2" - Current.

- '@types/react-router-dom': "^5.3.3" - react-router-dom is at 7.x, so types for 5.x are outdated.
- 'axios': "^1.12.2" - Current.
- 'react': "^19.2.0" - Current.
- 'react-dom': "^19.2.0" - Current.
- 'react-router-dom': "^7.9.4" - Current.
- 'react-scripts': "5.0.1" - react-scripts 5 is current, but Create React App is deprecated.
- 'recharts': "^3.5.1" - Current.
- 'typescript': "^4.9.5" - TypeScript is at 5.x, so 4.9.5 is outdated.
- 'web-vitals': "^2.1.4" - Current.

3. 'server/package.json':

- 'axios': "^1.13.2" and 'axios': "^1.6.7" - Duplicate dependency! This is a clear issue.
- 'bcryptjs': "^2.4.3" - Current.
- 'cors': "^2.8.5" - Current.
- 'dotenv': "^16.3.1" - Current.
- 'express': "^4.18.2" - Current.
- 'express-validator': "^7.0.1" - Current.
- 'firebase-admin': "^11.11.0" - firebase-admin is at 12.x, so 11.x is slightly outdated but not critical.
- 'helmet': "^7.1.0" - Current.
- 'morgan': "^1.10.0" - Current.
- 'jest': "^29.7.0" - Current.
- 'nodemon': "^3.0.1" - Current.
- 'supertest': "^6.3.3" - Current.

4. 'LOCAL_SETUP.md':

- Line 22-24: "Root 'npm test' is broken as inherited: 'tests/example.test.js' imports 'src/utills/businessLogic.js', which was never committed upstream. Per the assignment ("report, do not repair") we leave it as-is and report it." - This is explicit evidence of broken tests.

5. 'server/config/firebase.js':

- Lines 5-14: Hardcoded service account structure relying on environment variables. If these env vars are missing or expired, the app fails.
- Line 20: 'if (process.env.FIRESTORE_EMULATOR_HOST)' - Relies on emulator host env var.

6. 'server/routes/voice.js':

- Line 44: 'const model = process.env.GEMINI_MODEL || 'gemini-2.5-flash';' - 'gemini-2.5-flash' might not exist yet or might be deprecated.

KP3 ? Build failure triage

Prompt

""text

You are a senior engineer triaging a build failure.

Here is the exact error I hit trying to run this project locally:

<paste the FirebaseAppError stack trace>

Classify this failure: (a) code rot, (b) my local setup, or (c) a real bug in the project. Give your confidence for each option.

Output format:

1. Classification + confidence (%) for each of the 3 options

2. Root cause, cited to file and line
3. The cheapest fix, in order of effort

Rules:

- Only use evidence from the repo (e.g. LOCAL_SETUP.md, config files) ?
do not assume how Firebase projects "normally" work.
- If the repo doesn't explain something, say so instead of guessing.

...

Result

<think>

The user is asking me to triage a build failure related to a 'FirebaseAppError' stack trace. However, the prompt says: "Here is the exact error I hit trying to run this project locally: <paste the FirebaseAppError stack trace>" but the actual stack trace is missing from the prompt. Wait, looking closely at the prompt, it literally says '<paste the FirebaseAppError stack trace>'. This is a placeholder. I need to infer the likely error based on the repository snapshot, or point out that the stack trace is missing but deduce the most probable cause from the repo files.

Wait, the prompt says: "Here is the exact error I hit trying to run this project locally:

<paste the FirebaseAppError stack trace>"

This implies the user forgot to paste it, or it's a template. I will assume the error is related to Firebase initialization, likely 'FirebaseAppError: The default Firebase app does not exist.' or 'FirebaseAppError: Another app with name '[DEFAULT]' has already been initialized.' or something about missing credentials.

Let's look at 'server/config/firebase.js':

```
""javascript
const admin = require('firebase-admin');
require('dotenv').config();

// Initialize Firebase Admin SDK
const serviceAccount = {
  type: "service_account",
  project_id: process.env.FIREBASE_PROJECT_ID,
  private_key_id: process.env.FIREBASE_PRIVATE_KEY_ID,
  private_key: process.env.FIREBASE_PRIVATE_KEY?.replace(/\\n/g, '\n'),
  client_email: process.env.FIREBASE_CLIENT_EMAIL,
  client_id: process.env.FIREBASE_CLIENT_ID,
  auth_uri: process.env.FIREBASE_AUTH_URI,
  token_uri: process.env.FIREBASE_TOKEN_URI,
};

// Initialize Firebase Admin
// When FIRESTORE_EMULATOR_HOST is set, skip service-account credentials ?
// the Admin SDK connects to the local emulator with just a project id.
if (!admin.apps.length) {
  if (process.env.FIRESTORE_EMULATOR_HOST) {
    admin.initializeApp({
      projectId: process.env.FIREBASE_PROJECT_ID,
    });
  } else {
    admin.initializeApp({
      credential: admin.credential.cert(serviceAccount),
```

```

    projectId: process.env.FIREBASE_PROJECT_ID,
  });
}
}
}
...

```

And 'LOCAL_SETUP.md':

```
...
```

The original repo needs a real Firebase project to run. We run it locally against the Firestore emulator in Docker instead. Steps after cloning:

```

cd proj2
cp server/env.example server/.env # emulator config is the default
docker compose up -d             # Firestore emulator on :8080 (needs Docker)
npm run install-all              # root + server + client deps
npm run dev                      # server :5001 + client :3000
...

```

If the user runs 'npm run dev' without setting up the '.env' file or without running 'docker compose up -d', 'process.env.FIRESTORE_EMULATOR_HOST' will be undefined. Then it falls into the 'else' block:

```

""javascript
  admin.initializeApp({
    credential: admin.credential.cert(serviceAccount),
    projectId: process.env.FIREBASE_PROJECT_ID,
  });
...

```

If 'server/.env' is missing or doesn't have the credentials, 'serviceAccount' will have 'undefined' values, causing 'FirebaseAppError' or 'FirebaseError' during initialization. Alternatively, if they run it multiple times without hot reload handling, 'admin.apps.length' check prevents double init, but if they restart the server, it's fine.

Another possibility: 'FirebaseAppError: The default Firebase app does not exist. Call initializeApp() first.' This happens if 'admin.initializeApp()' fails or isn't called. But here it is called.

Wait, the prompt says '<paste the FirebaseAppError stack trace>'. I will treat it as a placeholder and analyze the likely Firebase initialization failure based on the repo. The most common 'FirebaseAppError' in this setup is due to missing environment variables or not using the emulator as instructed.

Let's classify:

- (a) code rot: Low confidence. The code explicitly handles emulator vs prod.
- (b) my local setup: High confidence. The 'LOCAL_SETUP.md' explicitly says to 'cp server/env.example server/.env' and 'docker compose up -d'. Skipping these leads to missing 'FIRESTORE_EMULATOR_HOST' or missing credentials, triggering Firebase init errors.
- (c) a real bug: Low confidence. The code structure is standard for Firebase Admin SDK with emulator support.

Root cause: Missing '.env' configuration or missing Docker emulator startup, causing 'process.env.FIRESTORE_EMULATOR_HOST' to be undefined, which forces the code to attempt production initialization with undefined credentials ('server/config/firebase.js', lines 24-28).

Cheapest fix:

1. Run 'cp server/env.example server/.env' (if not done).
2. Run 'docker compose up -d' to start the emulator.
3. Ensure 'FIRESTORE_EMULATOR_HOST=localhost:8080' is in '.env'.
4. Restart the dev server.

Wait, is there any other clue? 'LOCAL_SETUP.md' line 8: 'cp server/env.example server/.env # emulator config is the default'

Line 9: 'docker compose up -d # Firestore emulator on :8080 (needs Docker)'

If the user didn't do this, 'FIRESTORE_EMULATOR_HOST' is not set. The code at 'server/config/firebase.js:20' checks 'if (process.env.FIRE

KP4 ? Main feature check (voice ordering)

Prompt

""text

You are a QA engineer evaluating one feature end to end.

The main feature to evaluate is: voice ordering (/api/voice).

Does the code actually implement this well? Walk through the main scenario end to end and identify any place it would break or behave unexpectedly.

Output format:

1. Actor, trigger, and main flow (numbered steps, 6 max)
2. Does it work as intended? Yes/Partial/No, with evidence
3. Three weaknesses or missing pieces, each cited to file and line

Rules:

- Cite file and line for every claim about behavior.
 - If a step's behavior is unclear from the code, write "UNVERIFIED" rather than assuming standard behavior.
- ...

Result

<think>

Here's a thinking process:

1. **Understand User Goal**: The user wants a QA evaluation of the "voice ordering (/api/voice)" feature based on the provided repository snapshot. I need to:
 - Walk through the main scenario end-to-end (max 6 steps).
 - State if it works as intended (Yes/Partial/No) with evidence.
 - Identify 3 weaknesses/missing pieces, citing file and line.
 - Follow strict rules: cite file/line for every claim, use "UNVERIFIED" if unclear, don't assume standard behavior if not in code.
2. **Analyze the Repository Snapshot**:
 - **Server Side**: 'server/routes/voice.js'
 - Defines 'ACTIONS' array with IDs: 'logout', 'openProfile', 'goHome', 'openCart', 'calculateTotalPrice'.
 - 'POST /classify' endpoint.
 - Takes 'userText', builds a prompt for Gemini API.
 - Calls Gemini API ('gemini-2.5-flash' or env var).

- Parses response, checks if 'actionId' is in 'ACTION_SET'.
- Returns '{ actionId }' or error.
- ****Client Side****:
 - 'client/src/features/voice/api/gemini.ts': Calls '/voice/classify', validates response against 'getActionById'.
 - 'client/src/features/voice/utils/actions.ts': Defines 'VoiceActionId' type and 'ACTIONS' array. Matches server.
 - 'client/src/features/voice/utils/performAction.ts': Switch statement to execute actions based on 'actionId'.
 - 'client/src/features/voice/hooks/useSpeechToText.ts': Uses Web Speech API ('webkitSpeechRecognition'/'SpeechRecognition'). Handles start/stop, interim/final transcripts. (Truncated at line 99, but enough to see setup).
 - Other voice files exist but aren't fully shown: 'VoiceCommandManager.tsx', 'FloatingVoiceButton.tsx', 'VoiceModal.tsx', 'ConfirmationModal.tsx'. I'll note truncation/UNVERIFIED where needed.

3. ****Map the Main Flow (End-to-End)****:

1. User clicks voice button / triggers listening.
2. Browser's Web Speech API captures audio and converts to text ('useSpeechToText').
3. Final transcript is sent to backend '/voice/classify'.
4. Backend sends text to Gemini API for intent classification.
5. Backend returns classified 'actionId' to frontend.
6. Frontend executes the corresponding action via 'performAction'.

Let's refine to max 6 steps with citations:

1. User activates voice input via UI component (UNVERIFIED - 'FloatingVoiceButton.tsx'/'VoiceCommandManager.tsx' not fully shown, but 'useSpeechToText' hook exists at 'client/src/features/voice/hooks/useSpeechToText.ts:55').
2. Browser's Speech Recognition API captures audio and emits a final transcript ('client/src/features/voice/hooks/useSpeechToText.ts:55-99').
3. Frontend sends transcript to 'POST /voice/classify' ('client/src/features/voice/api/gemini.ts:10').
4. Backend constructs a prompt and calls Gemini API to classify intent ('server/routes/voice.js:48-65').
5. Backend validates the returned 'actionId' against a predefined set and responds ('server/routes/voice.js:71-77').
6. Frontend receives 'actionId', validates it, and dispatches the action via 'performAction' ('client/src/features/voice/utils/performAction.ts:11-32').

4. ****Evaluate "Does it work as intended?"****:

- The feature is labeled "voice ordering", but the available actions are: 'logout', 'openProfile', 'goHome', 'openCart', 'calculateTotalPrice'. None of these actually *place* an order or add items to the cart. They are navigation/utility actions.
- The flow is technically complete for classification and execution, but it doesn't fulfill "ordering".
- I'll rate it ****Partial****. Evidence: The action list lacks any ordering-related intents (e.g., 'addToCart', 'placeOrder', 'searchMenu'). The system only handles navigation and cart calculation. ('server/routes/voice.js:6-12', 'client/src/features/voice/utils/actions.ts:8-14').

5. ****Identify 3 Weaknesses/Missing Pieces****:

- ****Weakness 1: Missing Ordering Functionality****. The feature is named "voice ordering" but only supports navigation/utility actions. No action handles adding items, searching menus, or placing orders. ('server/routes/voice.js:6-12', 'client/src/features/voice/utils/actions.ts:8-14')

- **Weakness 2: Hardcoded/Static Action List & Prompt Injection Risk**. The prompt is built by concatenating user text directly into the string without sanitization beyond whitespace trimming. While Gemini is robust, it's a minor security/robustness concern. More importantly, the action list is duplicated between client and server, risking drift. ('server/routes/voice.js:18-28', 'client/src/features/voice/utls/actions.ts:8-14')

- **Weakness 3: Lack of Error Handling/Fallback for Speech Recognition**. The 'useSpeechToText' hook handles 'on

KP5 ? Existing test coverage for that feature

Prompt

""text

You are a QA lead reviewing test coverage for one feature.

Are there any existing tests for the voice ordering feature (/api/voice)? Evaluate their quality and identify untested edge cases.

Output format:

1. List of existing tests that touch this feature (file + what they check), or "NONE FOUND"
2. Quality verdict: strong / weak / missing, with reasoning
3. Any edge cases that are not currently tested

Rules:

- Only count a test as covering this feature if you can point to the exact assertion that checks it.
 - Do not infer coverage from a test's filename alone.
- ...

Result

<think>

The user wants to know about existing tests for the voice ordering feature ('/api/voice').

I need to check the repository snapshot for tests related to voice ordering.

The repository snapshot contains 'tests/example.test.js'.

Let's look at 'tests/example.test.js' to see if it covers voice ordering.

The file imports functions from './src/utls/businessLogic'.

It tests points calculation, order management, user roles, etc.

I will scan the content of 'tests/example.test.js' for any mention of "voice", "speech", "gemini", "api/voice", or related functionality.

The provided content of 'tests/example.test.js' is truncated, but I can see the first ~260 lines.

It covers:

- Points System (lines 36-148)
- Order Management (lines 151-260+)

I don't see any mention of voice ordering in the visible part.

Let's check the rest of the file list for other test files.

Only 'tests/example.test.js' is listed in the file list.

Wait, the prompt says "REPOSITORY FILE LIST: ... tests/example.test.js".

So there is only one test file.

I will search the provided text of 'tests/example.test.js' for "voice".

There is no mention of "voice" in the provided snippet.

The snippet ends at line 261.

The prompt says "The file list is complete, but file contents may be omitted due to the API

context limit; mark omitted evidence UNVERIFIED."

Since 'tests/example.test.js' is the only test file, and its content is truncated, I should state that based on the visible content, there are no tests for voice ordering. I will mark the rest of the file as UNVERIFIED if needed, but I can safely say "NONE FOUND" for the visible part, and note that the rest is truncated.

Wait, the prompt says "Only count a test as covering this feature if you can point to the exact assertion that checks it."

Since I don't see any, I will report NONE FOUND.

I will also evaluate quality as "missing" because there are no tests for this feature.

Edge cases: I will list typical edge cases for voice ordering (e.g., speech recognition errors, network failures, ambiguous commands, unsupported languages, API rate limits, etc.).

Let's double check the file list for any other test files.

'tests/example.test.js' is the only one.

'client/src/App.test.tsx' exists but its content is not provided. I should mention it but note it's UNVERIFIED.

The prompt asks specifically about '/api/voice'.

I will structure the output as requested.

1. List of existing tests: NONE FOUND (based on visible content of 'tests/example.test.js'; 'client/src/App.test.tsx' exists but content is UNVERIFIED).
2. Quality verdict: missing. Reasoning: No test assertions reference voice ordering, speech-to-text, or the '/api/voice' endpoint. The visible test suite focuses exclusively on business logic (points, order totals, status transitions).
3. Edge cases not tested: (list them)

Wait, let's verify if 'tests/example.test.js' might have voice tests in the truncated part. The prompt says "mark omitted evidence UNVERIFIED". I will state that the rest of the file is truncated/UNVERIFIED, but based on available evidence, none are found.
I will follow the exact output format.

Structure:

1. List of existing tests that touch this feature: NONE FOUND
2. Quality verdict: missing, with reasoning
3. Any edge cases that are not currently tested

I will draft the response carefully.

Check rules: "Only count a test as covering this feature if you can point to the exact assertion that checks it." -> NONE FOUND.

"Do not infer coverage from a test's filename alone." -> Acknowledged.

"mark omitted evidence UNVERIFIED" -> I will note that 'tests/example.test.js' is truncated and the remainder is UNVERIFIED, but no voice-related tests are visible.

Let's construct the response.

1. List of existing tests: NONE FOUND. (Note: 'tests/example.test.js' is the only test file provided, but its content is truncated after line 261. The visible portion covers points calculation and order management logic. The remainder is UNVERIFIED. 'client/src/App.test.tsx' exists but its content is UNVERIFIED.)
2. Quality verdict: missing. Reasoning: There are no visible test cases or assertions targeting the voice ordering feature, speech-to-text hooks, or the '/api/voice' endpoint. The existing test suite focuses entirely on deterministic business logic (points, discounts, order validation, status transitions) and does not include integration or unit tests for the voice feature.
3. Edge cases not tested:
 - Speech recognition confidence thresholds (handling low-confidence transcriptions)

- Network timeouts or failures during voice API calls
- Ambiguous or unrecognized commands (fallback behavior)
- Concurrent voice commands or rapid successive triggers
- Unsupported languages or accents
- API rate limiting or quota exhaustion for the Gemini/voice service
- Invalid or malformed JSON responses from the voice API
- User permissions/role checks for voice ordering (e.g., non-customers attempting to use it)
- Handling of special characters or punctuation in transcribed text
- Graceful degradation when the Web Speech API is unavailable in the browser

This matches the requirements. I will output it exactly.

One minor thing: The prompt says "Are there any existing tests for the voice ordering feature (/api/voice)?". I will stick

KP6 ? Write a test case

Prompt

""text

You are a senior engineer writing tests for naked code.

Write a test case for the voice ordering feature's main success path (the step described in KP4's main flow).

Output format:

- Test name (as a sentence, e.g. "test_rejects_empty_voice_input")
- The test code itself
- One line: "This proves ..."

Rules:

- Base the test only on the actual function signature and behavior you find in the code ? do not invent parameters or return values.
 - If you must assume something about the environment (e.g. a mocked Firebase call), state the assumption explicitly instead of hiding it.
- ...

Result

<think>

The user wants a test case for the voice ordering feature's main success path.

Looking at the repository, the voice feature is implemented in:

- 'server/routes/voice.js' (backend route for '/voice/classify')
- 'client/src/features/voice/api/gemini.ts' (frontend API call)
- 'client/src/features/voice/utils/actions.ts' (action definitions)
- 'client/src/features/voice/utils/performAction.ts' (action execution)
- 'client/src/features/voice/hooks/useSpeechToText.ts' (speech-to-text hook)

The prompt mentions "KP4's main flow". I don't have KP4 explicitly, but the main success path for voice ordering/classification seems to be:

1. User speaks -> 'useSpeechToText' gets transcript.
2. Transcript sent to '/voice/classify' via 'classifyWithGemini'.
3. Backend calls Gemini API, gets action ID.
4. Frontend receives action ID, validates it, and calls 'performAction'.

Let's focus on the core classification logic in 'server/routes/voice.js' or the frontend

'classifyWithGemini' function. The prompt says "voice ordering feature's main success path".

Actually, 'server/routes/voice.js' handles the classification.

Let's write a test for 'server/routes/voice.js' POST '/classify' endpoint, or maybe

'classifyWithGemini' in the client.

Given it's a backend route, testing it with 'supertest' or mocking 'axios' is standard.

Wait, the prompt says "Base the test only on the actual function signature and behavior you find in the code".

Let's look at 'server/routes/voice.js':

```
"""javascript
router.post('/classify', async (req, res) => {
  try {
    const { userText } = req.body || {};
    if (!userText || typeof userText !== 'string') {
      return res.status(400).json({ error: 'userText is required.' });
    }
    // ... calls axios to Gemini ...
    // returns res.json({ actionId });
  } catch (error) { ... }
});
"""
```

I will write a test for the '/classify' endpoint using 'supertest' and mocking 'axios'.

Alternatively, I could test 'classifyWithGemini' from

'client/src/features/voice/api/gemini.ts'.

Let's stick to the backend route 'server/routes/voice.js' as it's the core logic. Or maybe the frontend 'classifyWithGemini' is simpler.

Actually, the prompt says "voice ordering feature's main success path". I'll test the backend route '/classify' because it contains the prompt building and Gemini call.

Let's draft a test for 'server/routes/voice.js':

```
"""javascript
const request = require('supertest');
const express = require('express');
const axios = require('axios');
const voiceRouter = require('./server/routes/voice');
```

```
const app = express();
app.use(express.json());
app.use('/voice', voiceRouter);
```

```
jest.mock('axios');
```

```
describe('Voice Classification Route', () => {
  it('classifies valid user text and returns the correct action id', async () => {
    // Arrange
    const mockGeminiResponse = {
      data: {
        candidates: [{
          content: {
            parts: [{ text: 'openCart' }]
          }
        }]
      }
    };
    axios.post.mockResolvedValue(mockGeminiResponse);
```

```

process.env.GEMINI_API_KEY = 'test-key';

// Act
const res = await request(app)
  .post('/voice/classify')
  .send({ userText: 'Show me my cart' });

// Assert
expect(res.status).toBe(200);
expect(res.body.actionId).toBe('openCart');
expect(axios.post).toHaveBeenCalledWith(
  expect.stringContaining('/models/gemini-2.5-flash:generateContent'),
  expect.objectContaining({
    contents: [{ role: 'user', parts: [{ text: expect.stringContaining('Show me my cart')
  }} ]
  }
  )),
  expect.objectContaining({ params: { key: 'test-key' } })
);
});
});
"""

```

Wait, the prompt says "Base the test only on the actual function signature and behavior you find in the code ? do not invent parameters or return values."

The code in 'server/routes/voice.js' expects 'req.body.userText'.

It returns '{ actionId }'.

It uses 'axios.post'.

This matches.

Let's refine the test to be self-contained and follow the requested format.

Test name: "test_classifies_valid_voice_input_and_returns_action_id"

Code: Jest test using supertest and mocked axios.

Assumption: I will assume 'axios' is mocked and 'process.env.GEMINI_API_KEY' is set.

Let's check 'client/src/features/voice/api/gemini.ts' as well. It calls

'api.post('/voice/classify', { userText })'.

Maybe testing the frontend function is cleaner? But backend route is fine.

I'll stick to the backend route test.

Wait, the prompt says "KP4's main flow". I don't know what KP4 is, but it likely refers to a key point or requirement in their spec. I'll just focus on the main success path of the voice classification feature.

Let's write the test carefully.

```

"""javascript
const request = require('supertest');
const express = require

```

KP7 ? Gap analysis + test case for the gap

Prompt

```

"""text
You are a QA lead closing coverage gaps.

```

Based on KP5's findings, pick the single biggest untested edge case for the voice ordering feature. Explain why it's risky, then write a test case for it.

Output format:

1. The gap, in one sentence
2. Why it's risky (what could go wrong for a real user)
3. Test case: name, code, and "This proves ..." line

Rules:

- The gap must come from KP5's actual output, not a new guess.
- If no real gap was found in KP5, say so rather than inventing one.
- ...

Result

<think>

The user wants me to act as a QA lead closing coverage gaps based on KP5's findings.

KP5's findings (from the prompt) state:

- Existing tests: NONE FOUND for voice ordering.
- Quality verdict: missing.
- Edge cases not tested: Speech recognition confidence, network timeouts, ambiguous commands, concurrent commands, unsupported languages, API rate limiting, invalid JSON responses, user permissions, special characters, graceful degradation.

I need to pick the single biggest untested edge case from KP5's list.

"Network timeouts or failures during voice API calls" or "Ambiguous or unrecognized commands (fallback behavior)" or "Graceful degradation when the Web Speech API is unavailable" are strong candidates.

Given the code in 'server/routes/voice.js' and 'client/src/features/voice/api/gemini.ts', the system relies heavily on an external API (Gemini) and the Web Speech API.

A "network timeout or failure during voice API calls" is a classic and high-risk edge case for any feature dependent on external AI services. If the API hangs or fails, the user might be left in a limbo state (listening indefinitely, or getting a confusing error).

Another strong candidate is "Ambiguous or unrecognized commands". The code in 'server/routes/voice.js' line 71 checks 'if (!actionId || !ACTION_SET.has(actionId))'. If the AI returns something unexpected, it returns a 422 error. The client handles this in 'gemini.ts' line 24-26.

Let's pick "Network timeouts or failures during voice API calls" as it's explicitly listed in KP5 and is critical for UX in voice features.

Wait, KP5 listed: "Network timeouts or failures during voice API calls".

I will choose this one.

Why it's risky: If the voice API hangs or fails, the user might be left with the microphone active indefinitely, or receive a generic error that doesn't guide them to retry, leading to frustration and potential battery drain.

Test case: I need to write a test case. Since the repo uses Jest (seen in 'jest.config.js' and 'tests/example.test.js'), I'll write a Jest test.

The test should mock the API call to simulate a timeout or failure and verify the client handles it gracefully (e.g., stops listening, shows error).

Looking at 'client/src/features/voice/api/gemini.ts', it calls 'api.post('/voice/classify', { userText })'.

I can write a unit test for 'classifyWithGemini' that mocks 'api.post' to reject/timeout and

checks if the error is thrown/handled correctly.

Or I can write a test for the hook 'useSpeechToText' or the component.

Given the constraints, a unit test for the API wrapper is cleanest.

Let's draft the test case.

Name: 'should handle network timeout during voice classification gracefully'

Code:

```
""javascript
import { classifyWithGemini } from '../client/src/features/voice/api/gemini';
import api from '../client/src/services/api';

jest.mock('../client/src/services/api');

describe('Voice API Classification', () => {
  it('should handle network timeout during voice classification gracefully', async () => {
    // Arrange
    const mockTimeoutError = new Error('Network Error: Timeout');
    mockTimeoutError.code = 'ECONNABORTED';
    api.post.mockRejectedValue(mockTimeoutError);

    // Act & Assert
    await expect(classifyWithGemini('order a pizza')).rejects.toThrow('Network Error:
Timeout');
    expect(api.post).toHaveBeenCalledTimes(1, { userText: 'order a pizza' });
  });
});
""
```

Wait, the prompt says "This proves ..." line.

I will adjust the test to match the repo's structure. The repo uses TypeScript in client, but tests are in JS. I'll stick to JS for the test file as per 'tests/example.test.js'.

Actually, I should make sure the test aligns with the code. 'classifyWithGemini' catches errors and throws a new Error with the message.

Line 24-26 in 'gemini.ts':

```
""typescript
} catch (error: any) {
  const message = error.response?.data?.error || error.message || 'Gemini classification
failed.';
  throw new Error(message);
}
""
```

So if 'api.post' rejects with 'mockTimeoutError', 'error.message' will be 'Network Error: Timeout'. It will throw 'new Error('Network Error: Timeout')'.

The test is valid.

Let's refine the output format.

1. The gap, in one sentence
2. Why it's risky (what could go wrong for a real user)
3. Test case: name, code, and "This proves ..." line

Gap: Network timeouts or failures during voice API calls are not tested.

Risk: If the external AI service hangs or drops the connection, the app may leave the microphone active indefinitely or crash, causing battery drain and a broken user experience.

Test case: ...

Check rules: "The gap must come from KP5's actual output, not a new guess." -> KP5 listed

"Network timeouts or failures during voice API calls".

"If no real gap was found in KP5, say so rather than inventing one." -> Gap exists.

I will construct the response carefully.

One detail: 'client/src/features/voice/api/gemini.ts' is the file. I'll reference it correctly.

The test code should be self-contained and clear.

I'll format it exactly as requested.

KP8 ? Feature value / long-run viability

Prompt

""text

You are a product-minded engineer assessing long-term value.

Is the voice ordering feature and the donation service useful and worth maintaining over the next few years? Consider adoption cost, fragility, and whether it solves a real user problem.

Output format:

1. Verdict: worth keeping / worth reworking / worth cutting
2. Three pieces of evidence from the code or docs that support the verdict
3. One risk that could make this feature costly to maintain long-term

Rules:

- Base the verdict on evidence in the repo (code complexity, external dependencies, docs) ? not on general opinions about voice features.
- Flag anything you're inferring versus anything you can point to directly.

...

Result

<think>

Here's a thinking process:

1. **Analyze User Prompt:**

- **Role:** Product-minded engineer assessing long-term value.
- **Question:** Is the voice ordering feature and the donation service useful and worth maintaining over the next few years? Consider adoption cost, fragility, and whether it solves a real user problem.
- **Output Format:**
 1. Verdict: worth keeping / worth reworking / worth cutting
 2. Three pieces of evidence from the code or docs that support the verdict
 3. One risk that could make this feature costly to maintain long-term
- **Rules:** Base verdict on repo evidence (code complexity, external dependencies, docs). Flag inferences vs. direct evidence.

2. **Scan Repository for Relevant Features:**

- **Voice Ordering/Feature:**
 - 'client/src/features/voice/' directory contains multiple files: 'api/gemini.ts', 'components/ConfirmationModal.tsx', 'FloatingVoiceButton.tsx', 'VoiceCommandManager.tsx', 'VoiceCommandStyles.css', 'VoiceModal.tsx', 'hooks/useSpeechToText.ts', 'utils/actions.ts', 'utils/performAction.ts'.

- 'server/routes/voice.js' handles classification via Gemini API.
- Actions defined: 'logout', 'openProfile', 'goHome', 'openCart', 'calculateTotalPrice'.

(Note: These are navigation/UI actions, *not* actual ordering actions like "add pizza to cart" or "place order".)

- Uses Web Speech API ('webkitSpeechRecognition'/'SpeechRecognition') on the client.
- Uses Google Gemini API ('gemini-2.5-flash') on the server for intent classification.

- ****Donation Service:****

- 'server/routes/donations.js' exists in the file list.
- No content provided for 'server/routes/donations.js' in the snapshot. I'll need to note it's present but content is omitted/UNVERIFIED, or infer from context if possible. Wait, the prompt says "file contents may be omitted due to the API context limit; mark omitted evidence UNVERIFIED." I should check if there's any other reference to donations. I don't see any client-side code for donations in the file list. Only 'server/routes/donations.js'.
- I'll focus primarily on the voice feature since I have concrete code for it, and mention the donation service's lack of client integration or documentation as evidence.

3. ****Evaluate Voice Feature (Evidence & Analysis):****

- ***Adoption Cost/Fragility:** Relies on browser Web Speech API (known to be inconsistent across browsers, requires HTTPS, often needs user permission, deprecated/changed in some environments). Also relies on an external LLM API (Gemini) for simple intent classification.

- ***Real User Problem:** The actions are just navigation ('goHome', 'openProfile', 'openCart', 'logout', 'calculateTotalPrice'). It doesn't actually handle ordering (e.g., "order a burger", "add to cart"). This suggests it's a demo/prototype feature rather than a production-ready ordering system.

- ***Code Complexity:** Moderate. Well-structured ('useSpeechToText' hook, 'performAction' switch, 'gemini.ts' API wrapper). But the reliance on an LLM for 5 hardcoded actions is overkill and introduces latency/cost.

- ***Direct Evidence:**

- 'client/src/features/voice/utills/actions.ts' lines 8-14: Only defines 5 navigation/UI actions, no actual ordering logic.

- 'server/routes/voice.js' lines 44-45: Uses 'gemini-2.5-flash' via external API for classification.

- 'client/src/features/voice/hooks/useSpeechToText.ts' lines 47-53: Relies on 'window.SpeechRecognition || window.webkitSpeechRecognition', which is browser-dependent and lacks standardization.

- ***Inference:** The feature is likely a prototype/demo because it only handles navigation, not actual ordering, and uses an expensive LLM for simple keyword matching.

4. ****Evaluate Donation Service (Evidence & Analysis):****

- ***Evidence:** 'server/routes/donations.js' exists in the file list, but no client-side components, contexts, or API calls reference it. No documentation mentions it.

- ***Inference:** It's likely an unfinished or orphaned backend route. Without client integration or docs, it has zero adoption value currently.

- ***Direct Evidence:** File list shows 'server/routes/donations.js' but no corresponding client files or docs.

5. ****Synthesize Verdict:****

- Given the voice feature's limited scope (navigation only, not ordering), reliance on fragile browser APIs + external LLM for simple tasks, and the donation service's apparent lack of client integration, the verdict should be ****worth cutting**** (or at least ****worth reworking**** if they want to keep the concept, but cutting is more aligned with "long-term value" given current state). I'll go with ****worth cutting**** because it doesn't solve a real ordering problem, uses over-engineered/fragile tech for simple tasks, and the donation service is orphaned.

- Wait, the prompt asks for one verdict for both. I'll frame it as ****worth cutting**** (or

****worth reworking**** if they want to salvage the concept, but cutting is stronger given the evidence). Let's stick with ****worth cutting**** due to low ROI, fragility, and lack of actual ordering functionality.

Let's refine the verdict to match